



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización y Arquitectura de Computadoras

Reporte - Construcción de Máquinas de Estados Usando Memorias

INTEGRANTES EQUIPO 8:

- Saldaña Navarrete Andrea - *Grupo Teoria:* 2
- Tepal Briseño Hansel Yael - *Grupo Teoria:* 1
- Ugartechea González Luis Antonio - *Grupo Teoria:* 1

NOMBRE DELPROFESOR: Ing. Diana Karen Pérez Pérez
GRUPO: 06
SEMESTRE: 2027-1

FECHA DE ENTREGA: 8 de octubre del 2026

Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando direccionamiento de memorias con el método de direccionamiento entrada-estado.

Introducción

En el ámbito del diseño digital, el método de direccionamiento entrada-estado modificado permite implementar cartas ASM almacenando el comportamiento lógico en una memoria ROM y delegando la evaluación de las condiciones a un arreglo de multiplexores externos. A diferencia del direccionamiento entrada-estado clásico, la dirección de memoria exacta a la que debe acceder el sistema en cada pulso de reloj se forma únicamente con los bits del estado presente de la máquina.

La palabra de memoria se divide en campos específicos: una porción de prueba que indica al selector de entradas qué variable evaluar, un par de campos de "liga" que guardan los posibles estados siguientes (falso o verdadero), y un par de campos destinados a las salidas correspondientes a cada condición. El dimensionamiento y la organización de la memoria dependen directamente de la cantidad de estados y de la suma de las longitudes de estos campos lógicos, logrando una reducción significativa en el número de localidades de memoria requeridas en comparación con otros métodos.

Desarrollo

1. Diseñe una carta ASM con las siguientes características:

- Estados: 8 o más
- Entradas: 4 (No es necesario utilizar una entrada auxiliar)
- Salidas: 4
  - 2 con lógica estándar
  - 2 con lógica negada
- Colocar al menos 3 salidas condiciones como en la práctica anterior.

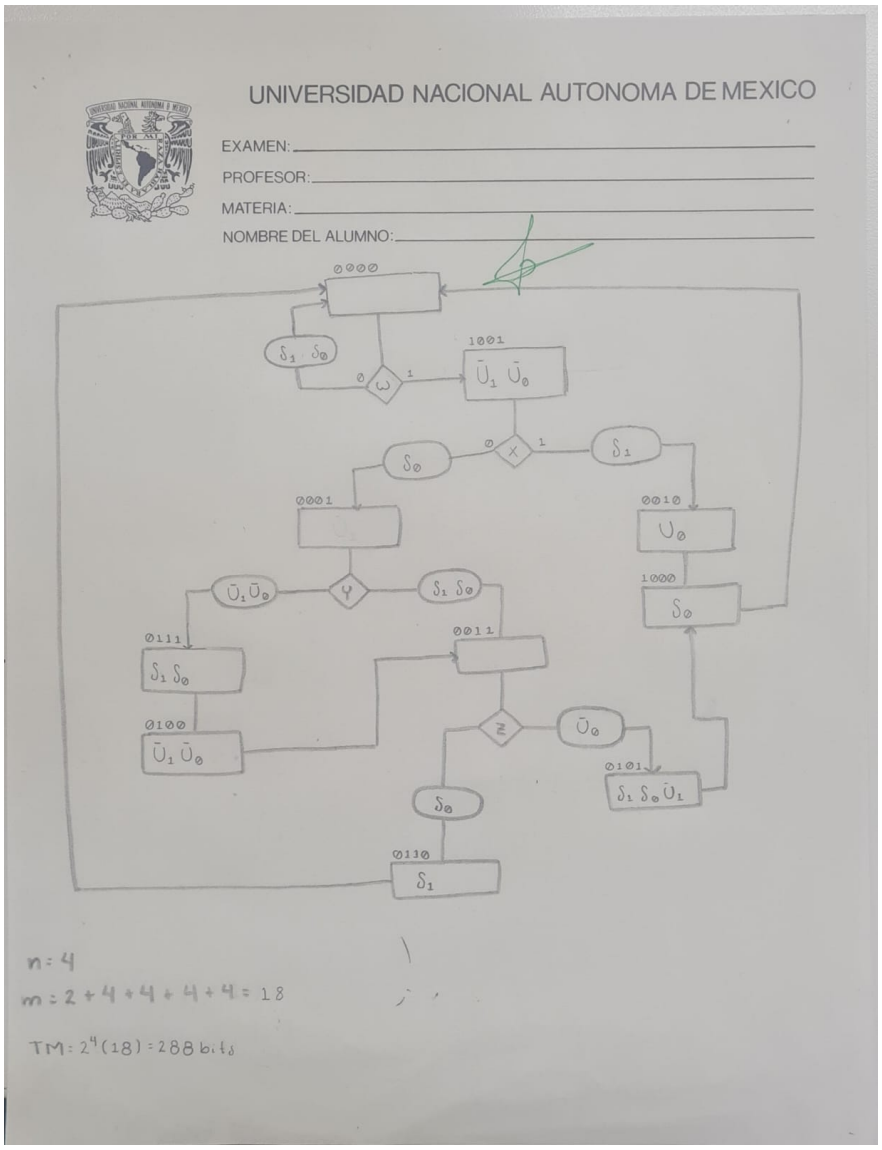


Figure 1: Carta ASM firmada por la profesora.

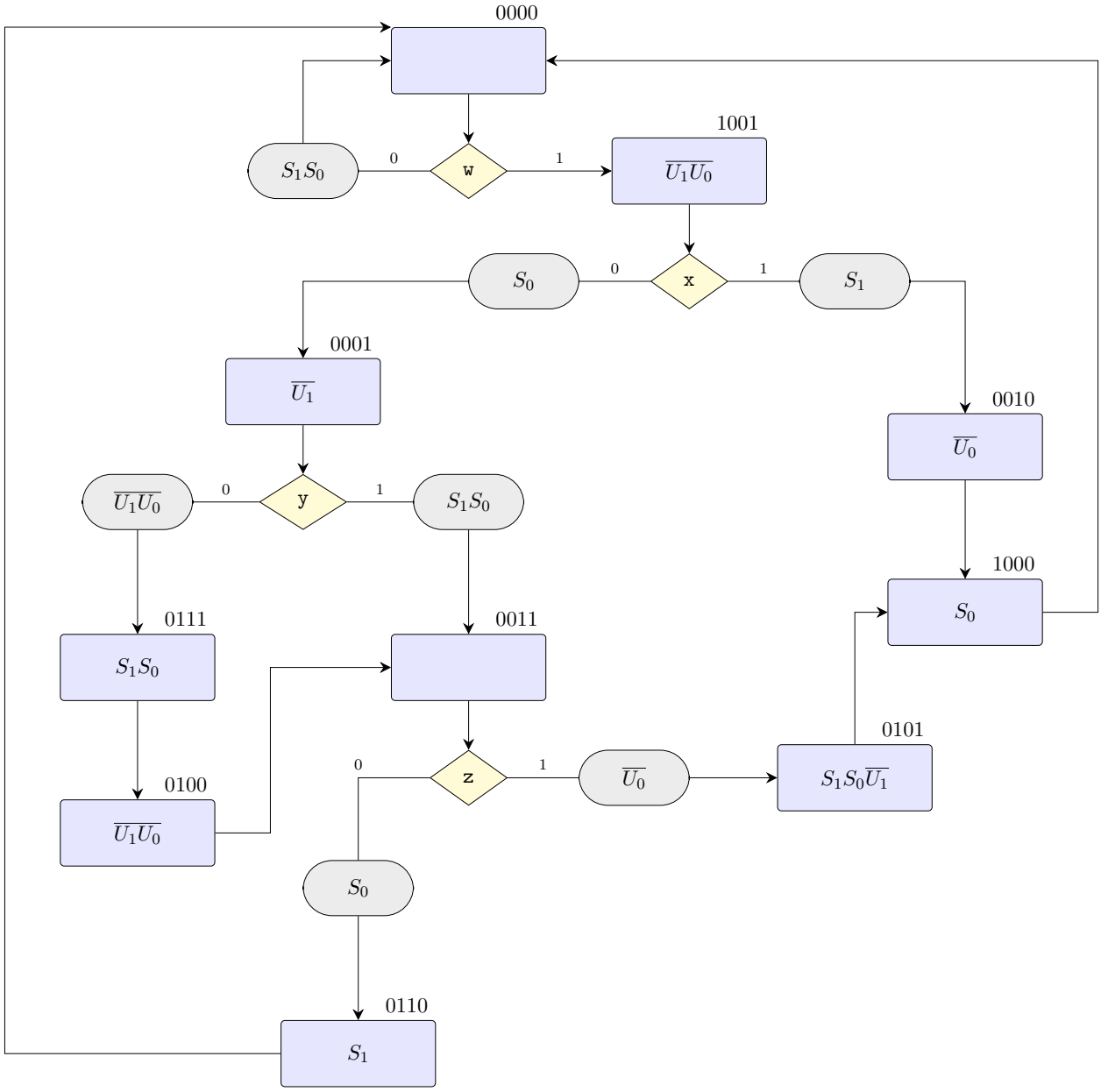


Figure 2: Carta ASM de 10 estados con enrutamiento limpio ortogonal (Direccionamiento Entrada-Estado).

2. Encuentre el tamaño de la memoria necesaria y el contenido de memoria utilizando el direccionamiento entrada-estado.

Cálculo del tamaño de memoria

Cabe tener en cuenta que para implementar salidas condicionales es necesario modificar el diagrama de bloques del direccionamiento entrada-estado.

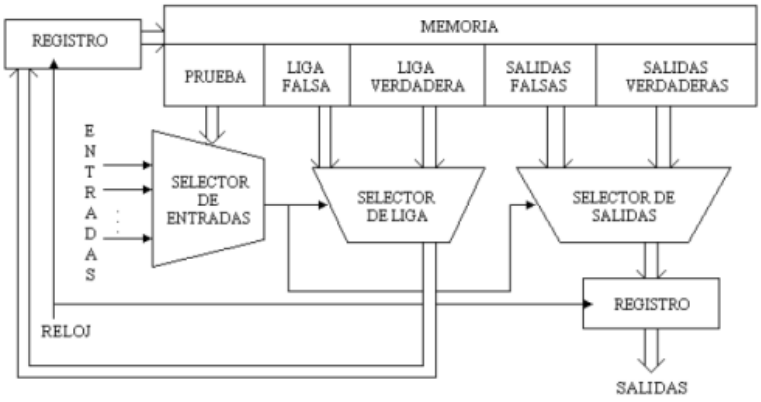


Figure 3: Esquema del Direccionamiento Entrada-Estado Modificado.

Para implementar la carta ASM mediante el direccionamiento entrada-estado modificado, la dirección de la memoria se forma únicamente con el estado presente. La palabra almacenada contiene el bit de prueba, los estados siguientes y las salidas correspondientes a la evaluación de la entrada.

Tamaño de memoria

$$2^n \cdot m$$

En donde:

- n = número de bits del estado presente.
- m = prueba + liga falsa + liga verdadera + salida falsa + salida verdadera.

En esta máquina se tienen 10 estados. Por lo tanto, se necesitan 4 bits para representarlos, ya que $2^4 = 16$ combinaciones posibles.

$$n = 4 \text{ bits}$$

La dirección de memoria tendrá 4 bits y podrá seleccionar 16 localidades. Para calcular la longitud de la palabra (m), se suman los bits de los campos requeridos por la arquitectura:

- **Prueba:** 2 bits (para seleccionar entre las entradas $w = 00, x = 01, y = 10, z = 11$).
- **Liga Falsa:** 4 bits (estado siguiente si la evaluación es 0).
- **Liga Verdadera:** 4 bits (estado siguiente si la evaluación es 1).

- **Salida Falsa:** 4 bits (estado de las salidas $S_1, S_0, \overline{U_1}, \overline{U_0}$ si la evaluación es 0).
- **Salida Verdadera:** 4 bits (estado de las salidas si la evaluación es 1).

$$m = 2 + 4 + 4 + 4 + 4 = 18 \text{ bits}$$

Por lo tanto, el tamaño de memoria necesario es:

$$2^4 \cdot 18 = 16 \cdot 18 = 288 \text{ bits}$$

Contenido de la memoria

El contenido de la memoria está formado por las 16 combinaciones posibles del estado presente. Al aplicar el concepto de **lógica mixta** (donde S_1 y S_0 encienden con `1`, mientras que $\overline{U_1}$ y $\overline{U_0}$ encienden con `0`), se definió que el estado inactivo base (nada encendido) corresponde al vector `0011`.

A continuación, se presenta la tabla de verdad completa (desplegada en 160 filas operativas), donde para cada estado se define qué ocurre con las rutas de liga y salidas bajo todas las variaciones de las entradas físicas.

Table 1: Tabla completa de transiciones y salidas de la máquina ASM (Lógica Mixta).

Estado Actual				Entradas				Estado Siguiente				Salidas			
$P_3(t)$	$P_2(t)$	$P_1(t)$	$P_0(t)$	w	x	y	z	$L_3(t+1)$	$L_2(t+1)$	$L_1(t+1)$	$L_0(t+1)$	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
0	0	0	0	0	0	0	1	0	0	0	0	1	1	1	1
0	0	0	0	0	0	1	0	0	0	0	0	1	1	1	1
0	0	0	0	0	0	1	1	0	0	0	0	1	1	1	1
0	0	0	0	0	1	0	0	0	0	0	0	1	1	1	1
0	0	0	0	0	1	0	1	0	0	0	0	1	1	1	1
0	0	0	0	0	1	1	0	0	0	0	0	1	1	1	1
0	0	0	0	0	1	1	1	0	0	0	0	1	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1	0	0	1	1
0	0	0	0	1	0	0	1	1	0	0	1	0	0	1	1
0	0	0	0	1	0	1	0	1	0	0	1	0	0	1	1
0	0	0	0	1	0	1	1	1	0	0	1	0	0	1	1
0	0	0	0	1	1	0	0	1	0	0	1	0	0	1	1
0	0	0	0	1	1	0	1	1	0	0	1	0	0	1	1
0	0	0	0	1	1	1	0	1	0	0	1	0	0	1	1
0	0	0	0	1	1	1	1	1	0	0	1	0	0	1	1
0	0	0	1	0	0	0	0	0	1	1	1	0	0	0	0
0	0	0	1	0	0	0	1	0	1	1	1	0	0	0	0
0	0	0	1	0	0	1	0	0	0	1	1	1	1	0	1
0	0	0	1	0	0	1	1	0	0	1	1	1	1	0	1
0	0	0	1	0	1	0	0	0	1	1	1	0	0	0	0
0	0	0	1	0	1	0	1	0	1	1	1	0	0	0	0
0	0	0	1	0	1	1	0	0	0	1	1	1	1	0	1
0	0	0	1	0	1	1	1	0	0	1	1	1	1	0	1
0	0	0	1	1	0	0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	1	0	1	1	1	0	0	0	0
0	0	0	1	1	0	1	0	0	0	1	1	1	1	0	1
0	0	0	1	1	0	1	1	0	0	1	1	1	1	0	1
0	0	0	1	1	1	0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	1	0	1	0	1	1	1	0	0	0	0
0	0	0	1	1	1	1	0	0	0	1	1	1	1	0	1
0	0	0	1	1	1	1	1	0	0	1	1	1	1	0	1
0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0
0	0	1	0	0	0	0	1	1	0	0	0	0	0	1	0
0	0	1	0	0	0	1	0	1	0	0	0	0	0	1	0
0	0	1	0	0	0	1	1	1	0	0	0	0	0	1	0
0	0	1	0	0	1	0	0	1	0	0	0	0	0	1	0
0	0	1	0	0	1	0	1	1	0	0	0	0	0	1	0
0	0	1	0	0	1	1	0	1	0	0	0	0	0	1	0
0	0	1	0	0	1	1	1	1	0	0	0	0	0	1	0
0	0	1	0	1	0	0	0	1	0	0	0	0	0	1	0

Estado Actual				Entradas				Estado Siguiente				Salidas			
$P_3(t)$	$P_2(t)$	$P_1(t)$	$P_0(t)$	w	x	y	z	$L_3(t+1)$	$L_2(t+1)$	$L_1(t+1)$	$L_0(t+1)$	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$
0	0	1	0	1	0	0	1	1	0	0	0	0	0	1	0
0	0	1	0	1	0	1	0	1	0	0	0	0	0	1	0
0	0	1	0	1	0	1	1	1	0	0	0	0	0	1	0
0	0	1	0	1	1	0	0	1	0	0	0	0	0	1	0
0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	0
0	0	1	0	1	1	1	0	1	0	0	0	0	0	1	0
0	0	1	0	1	1	1	1	1	0	0	0	0	0	1	0
0	0	1	1	0	0	0	0	0	1	1	0	0	1	1	1
0	0	1	1	0	0	0	1	0	1	0	1	0	0	1	0
0	0	1	1	0	0	1	0	0	1	1	0	0	1	1	1
0	0	1	1	0	0	1	1	0	1	0	1	0	0	1	0
0	0	1	1	0	1	0	0	0	1	1	0	0	1	1	1
0	0	1	1	0	1	0	1	0	1	0	1	0	0	1	0
0	0	1	1	0	1	1	0	0	1	1	0	0	1	1	1
0	0	1	1	0	1	1	1	0	1	0	1	0	0	1	0
0	0	1	1	1	0	0	0	0	1	1	0	0	1	1	1
0	0	1	1	1	0	0	1	0	1	0	1	0	0	1	0
0	0	1	1	1	0	1	0	0	1	1	0	0	1	1	1
0	0	1	1	1	0	1	1	0	1	0	1	0	0	1	0
0	0	1	1	1	1	0	0	0	1	1	0	0	1	1	1
0	0	1	1	1	1	0	1	0	1	0	1	0	0	1	0
0	0	1	1	1	1	1	0	0	1	1	0	0	1	1	1
0	0	1	1	1	1	1	1	0	1	0	1	0	0	1	0
0	1	0	0	0	0	0	0	0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1	0	0	1	1	0	0	0	0
0	1	0	0	0	0	1	0	0	0	1	1	0	0	0	0
0	1	0	0	0	0	1	1	0	0	1	1	0	0	0	0
0	1	0	0	0	1	0	0	0	0	1	1	0	0	0	0
0	1	0	0	0	1	0	1	0	0	1	1	0	0	0	0
0	1	0	0	0	1	1	0	0	0	1	1	0	0	0	0
0	1	0	0	0	1	1	1	0	0	1	1	0	0	0	0
0	1	0	0	1	0	0	0	0	0	1	1	0	0	0	0
0	1	0	0	1	0	0	1	0	0	1	1	0	0	0	0
0	1	0	0	1	0	1	0	0	0	1	1	0	0	0	0
0	1	0	0	1	0	1	1	0	0	1	1	0	0	0	0
0	1	0	0	1	1	0	0	0	0	1	1	0	0	0	0
0	1	0	0	1	1	0	1	0	0	1	1	0	0	0	0
0	1	0	0	1	1	1	0	0	0	1	1	0	0	0	0
0	1	0	0	1	1	1	1	0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0	1	0	0	0	1	1	0	1
0	1	0	1	0	0	0	1	1	0	0	0	1	1	0	1
0	1	0	1	0	0	1	0	1	0	0	0	1	1	0	1
0	1	0	1	0	0	1	1	1	0	0	0	1	1	0	1
0	1	0	1	0	1	0	0	1	0	0	0	1	1	0	1
0	1	0	1	0	1	0	1	1	0	0	0	1	1	0	1
0	1	0	1	0	1	1	0	1	0	0	0	1	1	0	1
0	1	0	1	0	1	1	1	1	0	0	0	1	1	0	1
0	1	0	1	1	0	0	0	1	0	0	0	1	1	0	1
0	1	0	1	1	0	0	1	1	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0	1	0	0	0	1	1	0	1
0	1	0	1	1	0	1	1	1	0	0	0	1	1	0	1
0	1	0	1	1	1	0	0	1	0	0	0	1	1	0	1
0	1	0	1	1	1	0	1	1	0	0	0	1	1	0	1
0	1	0	1	1	1	1	0	1	0	0	0	1	1	0	1
0	1	0	1	1	1	1	1	1	0	0	0	1	1	0	1
0	1	1	0	0	0	0	0	0	0	0	0	1	0	1	1
0	1	1	0	0	0	0	1	0	0	0	0	1	0	1	1
0	1	1	0	0	0	1	0	0	0	0	0	1	0	1	1
0	1	1	0	0	0	1	1	0	0	0	0	1	0	1	1
0	1	1	0	0	1	0	0	0	0	0	0	1	0	1	1
0	1	1	0	0	1	0	1	0	0	0	0	1	0	1	1
0	1	1	0	0	1	1	0	0	0	0	0	1	0	1	1
0	1	1	0	0	1	1	1	0	0	0	0	1	0	1	1
0	1	1	0	1	0	0	0	0	0	0	0	1	0	1	1
0	1	1	0	1	0	0	1	0	0	0	0	1	0	1	1
0	1	1	0	1	0	1	0	0	0	0	0	1	0	1	1
0	1	1	0	1	0	1	1	0	0	0	0	1	0	1	1
0	1	1	0	1	1	0	0	0	0	0	0	1	0	1	1
0	1	1	0	1	1	0	1	0	0	0	0	1	0	1	1
0	1	1	0	1	1	1	0	0	0	0	0	1	0	1	1
0	1	1	0	1	1	1	1	0	0	0	0	1	0	1	1
0	1	1	1	0	0	0	0	0	1	0	0	1	1	1	1
0	1	1	1	0	0	0	1	0	1	0	0	1	1	1	1
0	1	1	1	0	0	1	0	0	1	0	0	1	1	1	1
0	1	1	1	0	0	1	1	0	1	0	0	1	1	1	1
0	1	1	1	0	1	0	0	0	1	0	0	1	1	1	1
0	1	1	1	0	1	0	1	0	1	0	0	1	1	1	1
0	1	1	1	0	1	1	0	0	1	0	0	1	1	1	1
0	1	1	1	0	1	1	1	0	1	0	0	1	1	1	1
0	1	1	1	1	0	0	0	0	1	0	0	1	1	1	1
0	1	1	1	1	0	0	1	0	1	0	0	1	1	1	1
0	1	1	1	1	0	1	0	0	1	0	0	1	1	1	1
0	1	1	1	1	0	1	1	0	1	0	0	1	1	1	1
0	1	1	1	1	1	0	0	0	1	0	0	1	1	1	1
0	1	1	1	1	1	0	1	0	1	0	0	1	1	1	1
0	1	1	1	1	1	1	0	0	1	0	0	1	1	1	1
0	1	1	1	1	1	1	1	0	1	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
1	0	0	0	0	0	0	1	0	0	0	0	0	1	1	1
1	0	0	0	0	0	1	0	0	0	0	0	0	1	1	1
1	0	0	0	0	0	1	1	0	0	0	0	0	1	1	1
1	0	0	0	0	1	0	0	0	0	0	0	0	1	1	1

Estado Actual				Entradas				Estado Siguiente				Salidas			
$P_3(t)$	$P_2(t)$	$P_1(t)$	$P_0(t)$	w	x	y	z	$L_3(t+1)$	$L_2(t+1)$	$L_1(t+1)$	$L_0(t+1)$	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$
1	0	0	0	0	1	0	1	0	0	0	0	0	1	1	1
1	0	0	0	0	1	1	0	0	0	0	0	0	1	1	1
1	0	0	0	0	1	1	1	0	0	0	0	0	1	1	1
1	0	0	0	1	0	0	0	0	0	0	0	0	1	1	1
1	0	0	0	1	0	0	1	0	0	0	0	0	1	1	1
1	0	0	0	1	0	1	0	0	0	0	0	0	1	1	1
1	0	0	0	1	0	1	1	0	0	0	0	0	1	1	1
1	0	0	0	1	1	0	0	0	0	0	0	0	1	1	1
1	0	0	0	1	1	0	1	0	0	0	0	0	1	1	1
1	0	0	0	1	1	1	0	0	0	0	0	0	1	1	1
1	0	0	0	1	1	1	1	0	0	0	0	0	1	1	1
1	0	0	1	0	0	0	0	0	0	0	1	0	1	0	0
1	0	0	1	0	0	0	1	0	0	0	1	0	1	0	0
1	0	0	1	0	0	1	0	0	0	0	1	0	1	0	0
1	0	0	1	0	0	1	1	0	0	0	1	0	1	0	0
1	0	0	1	0	1	0	0	0	0	1	0	1	0	0	0
1	0	0	1	0	1	0	1	0	0	1	0	1	0	0	0
1	0	0	1	0	1	1	0	0	0	1	0	1	0	0	0
1	0	0	1	0	1	1	1	0	0	1	0	1	0	0	0
1	0	0	1	1	0	0	0	0	0	0	1	0	1	0	0
1	0	0	1	1	0	0	1	0	0	0	1	0	1	0	0
1	0	0	1	1	0	1	0	0	0	0	1	0	1	0	0
1	0	0	1	1	0	1	1	0	0	0	1	0	1	0	0
1	0	0	1	1	1	0	0	0	0	1	0	1	0	0	0
1	0	0	1	1	1	0	1	0	0	1	0	1	0	0	0
1	0	0	1	1	1	1	0	0	0	1	0	1	0	0	0
1	0	0	1	1	1	1	1	0	0	1	0	1	0	0	0

3. Una vez que haya obtenido el contenido de memoria, implemente el direccionamiento entrada-estado utilizando el software de desarrollo Quartus y escriba el contenido de memoria obtenido.

La implementación se realizó en Quartus Prime 18.1 Lite, seleccionando la familia MAX 10 y el dispositivo 10M50DCF484C7G. El archivo `practica_3.bdf` se utilizó como entidad de nivel superior; en él se conectaron los bloques que forman la ruta de datos de la máquina: el divisor de frecuencia, el registro de datos, un multiplexor de 4 a 1, dos multiplexores de 2 a 1, la memoria y un divisor de datos.

El flujo de trabajo del sistema opera mediante un ciclo de ejecución síncrono. En cada flanco de subida del reloj (previamente escalado), el registro actualiza su valor, estableciendo el "estado presente" de 4 bits. Este valor se utiliza como dirección (bus de direcciones) para acceder a una localidad específica en la memoria ROM. Al recibir esta dirección, la memoria devuelve una palabra larga de 18 bits que contiene toda la información de control para dicho estado: qué entrada debe evaluarse, cuáles son los posibles estados siguientes y cuáles son las posibles salidas.

Esta palabra de 18 bits pasa por un bloque divisor de datos que se encarga de separar (desempaquetar) la información en buses más pequeños y enviarla a los multiplexores. Primero, los bits de prueba dictan al multiplexor de 4:1 cuál de las entradas físicas (w, x, y, z) se debe evaluar en ese instante. El resultado de esta evaluación (un 0 o un 1) sirve como señal de selección simultánea para dos multiplexores de 2:1. El primer multiplexor de 2:1 selecciona el estado siguiente (liga verdadera o liga falsa) y lo retroalimenta a la entrada del registro para el próximo ciclo de reloj. El segundo multiplexor selecciona el arreglo de salidas correspondiente y lo dirige a los pines de salida de la FPGA.

Divisor de Frecuencia (`div_frec.vhd`)

Implementa un contador de 28 bits que incrementa su valor con cada pulso del reloj de la FPGA. Su propósito es dividir la alta frecuencia de operación para generar una señal de reloj lenta (`div_clk`) a partir del bit 25 de la cuenta. Esto permite que las transiciones de estado sean perceptibles a simple vista durante la demostración física.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity div_frec is
  Port (reloj : in std_logic;
        div_clk : out std_logic);
end div_frec;

architecture Behavioral of div_frec is
begin
  process (reloj)
  variable cuenta : std_logic_vector (27 downto 0) := X"0000000";
  begin
    if rising_edge (reloj) then

      --if cuenta 0 "4000000" then
      if cuenta = X"2FAF080" then
        cuenta := X"0000000";
      else
        cuenta := cuenta+1;
      end if;
    end if;
    div_clk <= cuenta(25);
  end process;
end Behavioral;
```

Listing 1: Código completo del divisor de frecuencia.

Registro de Estado (`register3.vhd`)

Funciona como la memoria volátil de la máquina, almacenando el estado actual. Está configurado para manejar un bus de 4 bits. En cada flanco positivo del reloj dividido, actualiza su salida (`DATA_OUT`) con el valor del estado siguiente (`DATA_IN`), el cual servirá como la nueva dirección de lectura para la memoria ROM. Cuenta también con un reinicio asíncrono para volver al estado "0000".

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```vhdl
entity register3 is
  Port( CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        DATA_IN : in STD_LOGIC_VECTOR(3 downto 0); --Liga
        DATA_OUT : out STD_LOGIC_VECTOR(3 downto 0)--Edo Presente
        );
end register3;

architecture Behavioral of register3 is
  signal internal_value : std_logic_vector(3 downto 0) := B"0000";
begin
  process (CLK, RESET, DATA_IN)
  begin
    if RESET = '0' then
      internal_value <= B"0000";
    elsif rising_edge (CLK) then
      internal_value <= DATA_IN;
    end if;
  end process;

  process(internal_value)
  begin
    DATA_OUT <= internal_value;
  end process;
end Behavioral;
```

Listing 2: Código completo del registro de estado.

Memoria ROM (`memory.vhd`)

Es el componente central del direccionamiento por trayectoria. Se estructuró como un arreglo de memoria constante de 16 localidades (direccionadas por los 4 bits del estado presente) con palabras de 18 bits de longitud. En ella se mapearon manualmente los campos lógicos correspondientes a los 10 estados de la carta ASM, dejando los 6 estados no utilizados en cero.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity memory is
  Port( dir : in STD_LOGIC_VECTOR( 3 downto 0); -- 4 bits de
     direcciones (n=4)
      data : out STD_LOGIC_VECTOR( 17 downto 0) -- 18 bits de
         memoria por registro (m=18)
      );
end memory;
```

```
11
12  architecture Behavioral of memory is
13    type mem is array(0 to 15) of std_logic_vector( 17 downto 0);
14
15    signal internal_mem : mem := (
16
17      -- ESTADO 0000: Evalúa 'w' (00). Base: Nada (0011)
18      -- Falso: S1, S0 ON (1111), Liga 0000.  Verdadero: Nada ON
            (0011), Liga 1001.
19      0 => B"00_0000_1001_1111_0011",
20
21      -- ESTADO 0001: Salida base U1' ON (0001). Evalúa 'y' (10).
22      -- Falso: U1', U0' ON (0000), Liga 0111.
23      -- Verdadero: Base + S1, S0 ON (1101), Liga 0011.
24      1 => B"10_0111_0011_0000_1101",
25
26      -- ESTADO 0010: Incondicional. Salida U0' ON (0010). Liga
            directa a 1000.
27      2 => B"00_1000_1000_0010_0010",
28
29      -- ESTADO 0011: Evalúa 'z' (11). Base: Nada (0011)
30      -- Falso: S0 ON (0111), Liga 0110. Verdadero: U0' ON (0010),
            Liga 0101.
31      3 => B"11_0110_0101_0111_0010",
32
33      -- ESTADO 0100: Incondicional. Salida U1', U0' ON (0000). Liga
            directa a 0011.
34      4 => B"00_0011_0011_0000_0000",
35
36      -- ESTADO 0101: Incondicional. Salida S1, S0, U1' ON (1101).
            Liga directa a 1000.
37      5 => B"00_1000_1000_1101_1101",
38
39      -- ESTADO 0110: Incondicional. Salida S1 ON (1011). Liga directa
             a 0000.
40      6 => B"00_0000_0000_1011_1011",
41
42      -- ESTADO 0111: Incondicional. Salida S1, S0 ON (1111). Liga
            directa a 0100.
43      7 => B"00_0100_0100_1111_1111",
44
45      -- ESTADO 1000: Incondicional. Salida S0 ON (0111). Liga directa
             a 0000.
46      8 => B"00_0000_0000_0111_0111",
47
48      -- ESTADO 1001: Salida base U1', U0' ON (0000). Evalúa 'x' (01).
49      -- Falso: Base + S0 ON (0100), Liga 0001.
50      -- Verdadero: Base + S1 ON (1000), Liga 0010.
```

```
    9 => B"01_0001_0010_0100_1000",

    -- ESTADOS SIN USO (10 al 15): Llenos con el estado de "Apagado"
        para todos los LEDs
    others => B"00_0000_0000_0011_0011"
  );

begin
  process(dir)
  begin
    data <= internal_mem(conv_integer(unsigned(dir)));
  end process;
end Behavioral;
```

Listing 3: Código completo de la memoria ROM.

Divisor de Datos (`divisor_datos.vhd`)

Es un bloque estructural de enrutamiento. Su único propósito es recibir la palabra completa de 18 bits desde la memoria y desempaquetarla en cinco buses independientes: Prueba (2 bits), Liga Falsa (4 bits), Liga Verdadera (4 bits), Salida Falsa (4 bits) y Salida Verdadera (4 bits).

```
    library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity divisor_datos is
  Port( entrada : in STD_LOGIC_VECTOR( 17 downto 0); -- 18 bits que
     vienen de la memoria

      Prueba           : out STD_LOGIC_VECTOR(1 downto 0); -- Para
         el MUX4 (2 bits)
      Liga_Falsa       : out STD_LOGIC_VECTOR(3 downto 0); -- Para
         el MUX2_3 (4 bits)
      Liga_Verdadera   : out STD_LOGIC_VECTOR(3 downto 0); -- Para
         el MUX2_3 (4 bits)
      Salida_Falsa     : out STD_LOGIC_VECTOR(3 downto 0); -- Para
         el MUX2_4 (4 bits)
      Salida_Verdadera : out STD_LOGIC_VECTOR(3 downto 0)  -- Para
         el MUX2_4 (4 bits)
      );
end divisor_datos;

architecture Behavioral of divisor_datos is
begin
  process (entrada)
  begin

```

```
    Prueba           <= entrada(17 downto 16);
    Liga_Falsa       <= entrada(15 downto 12);
    Liga_Verdadera   <= entrada(11 downto 8);
    Salida_Falsa     <= entrada(7 downto 4);
    Salida_Verdadera <= entrada(3 downto 0);

  end process;
end Behavioral;
```

Listing 4: Código completo del divisor de datos.

Multiplexor Selector de Entradas (`mux4.vhd`)

Implementa un multiplexor de 4 a 1 para señales de 1 bit. Utiliza el bus de `Prueba` (2 bits) como selector para conmutar entre cuatro condiciones lógicas de la carta ASM (representadas por las entradas I_0, I_1, I_2, I_3). La salida elegida dictamina si el flujo tomará el camino de la liga/salida falsa (0) o verdadera (1).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_4 is
  Port( sel : in STD_LOGIC;
      Salida_Falsa, Salida_Verdadera : in STD_LOGIC_VECTOR(3 downto
         0); -- Entradas de salidas
      O : out STD_LOGIC_VECTOR(3 downto 0)
      );
end mux2_4;

architecture Behavioral of mux2_4 is
begin
  process (sel, Salida_Falsa, Salida_Verdadera) -- Agregar variables
  begin
    if sel = '1' then
      O <= Salida_Verdadera;
    else
      O <= Salida_Falsa;
    end if;
  end process;
end Behavioral;
```

Listing 5: Código completo del multiplexor selector de entradas.

Multiplexores Selectores de Liga y Salida (`mux2_3.vhd` y `mux2_4.vhd`)

Ambos componentes implementan un comportamiento idéntico, actuando como multiplexores de 2 a 1 para buses de 4 bits, gobernados por el resultado del selector de entradas.

- **`mux2_3.vhd` (Liga):** Selecciona cuál será el estado siguiente de la máquina, eligiendo entre la `Liga_Falsa` o la `Liga_Verdadera`.
- **`mux2_4.vhd` (Salida):** Establece el estado de los indicadores luminosos de la FPGA, conmutando entre la `Salida_Falsa` o la `Salida_Verdadera`.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_3 is
  Port( sel : in STD_LOGIC;
      Liga_Falsa, Liga_Verdadera : in STD_LOGIC_VECTOR(3 downto 0);
         -- Ligas de 4 bits
      O : out STD_LOGIC_VECTOR(3 downto 0) -- Salida ajustada a 4
         bits
      );
end mux2_3;

architecture Behavioral of mux2_3 is
begin
  process (sel, Liga_Falsa, Liga_Verdadera) -- Agregar variables a
     la lista de sensibilidad
  begin
    if sel = '1' then
      O <= Liga_Verdadera;
    else
      O <= Liga_Falsa;
    end if;
  end process;
end Behavioral;
```

Listing 6: Código completo del multiplexor selector de liga.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_4 is
  Port( sel : in STD_LOGIC;
      Salida_Falsa, Salida_Verdadera : in STD_LOGIC_VECTOR(3 downto
         0); -- Entradas de salidas
      O : out STD_LOGIC_VECTOR(3 downto 0)
      );
end mux2_4;

architecture Behavioral of mux2_4 is
begin
  process (sel, Salida_Falsa, Salida_Verdadera) -- Agregar variables
  begin
    if sel = '1' then
```

```
16        O <= Salida_Verdadera;
17      else
18        O <= Salida_Falsa;
19      end if;
20    end process;
21  end Behavioral;
```

Listing 7: Código completo del multiplexor selector de salida.

Discusión de Resultados

5. Simule su diseño para probar su funcionamiento. Recuerden que en sus simulaciones debe aparecer el contenido de la memoria además del estado presente.

Simulación 1

Esta primera simulación se realizó con las entradas `W=0,X=0, Y=1, Z=1` como valor inicial. Y se modificaron las entradas para visualizar el ciclo más largo de la carta ASM, las entradas en cuestión son `W=1,X=0, Y=0, Z=0`. La secuencia de estados y salidas es la siguiente:

sec-1: Estado: `0000` → Salida: `1111` → Estado siguiente: `0000` →

sec-2: Estado: `0000` → Salida: `0011` → Estado siguiente: `1001` →

sec-3: Estado: `1001` → Salida: `0100` → Estado siguiente: `0001` →

sec-4: Estado: `0001` → Salida: `0000` → Estado siguiente: `0111` →

sec-5: Estado: `0111` → Salida: `1111` → Estado siguiente: `0100` →

sec-6: Estado: `0100` → Salida: `0000` → Estado siguiente: `0011` →

sec-7: Estado: `0011` → Salida: `0111` → Estado siguiente: `0110` →

sec-8: Estado: `0110` → Salida: `1011` → Estado siguiente: `0000` →

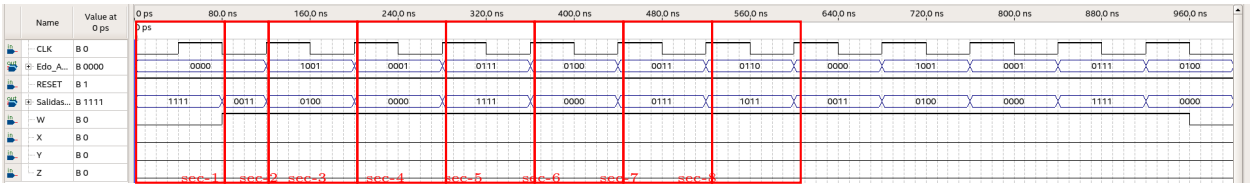


Figure 4: Simulación 1 de la máquina de estados en ModelSim, segmentada por transiciones de estado.

Se observa que la máquina de estados sigue correctamente la carta ASM, conmutando entre los estados y salidas esperadas según las condiciones de entrada.

Simulación 2

Para esta segunda simulación se parte de igual forma desde las entradas iniciales `W=0,X=0, Y=0, Z=0` y se modifican las entradas a `W=1,X=0, Y=1, Z=1` para visualizar la segunda secuencia más larga de la carta ASM. La secuencia de estados y salidas es la siguiente:

sec-1: Estado: `0000` → Salida: `1111` → Estado siguiente: `0000` →

sec-2: Estado: `0000` → Salida: `0011` → Estado siguiente: `1001` →

sec-3: Estado: `1001` → Salida: `0100` → Estado siguiente: `0001` →

sec-4: Estado: `0001` → Salida: `1101` → Estado siguiente: `0011` →

sec-5: Estado: `0011` → Salida: `0100` → Estado siguiente: `0101` →

sec-6: Estado: `0101` → Salida: `1101` → Estado siguiente: `1000` →

sec-7: Estado: `1000` → Salida: `0111` → Estado siguiente: `0000` →

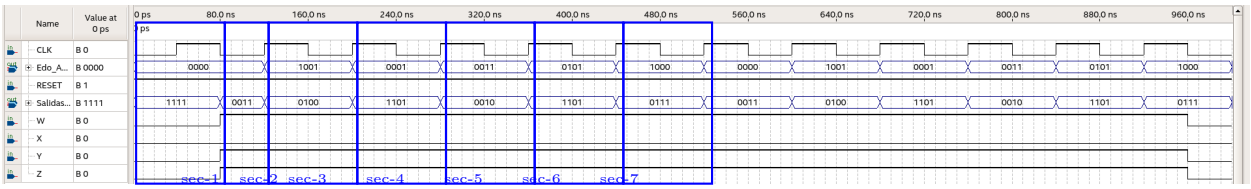


Figure 5: Simulación 1 de la máquina de estados en ModelSim, segmentada por transiciones de estado.

En este camnio exploramos a su vez dos posibles ciclos tomando como `Y = 1, Z = 1`, de tal forma que la máquina nos guíe a un camino diferente al que vimos en la primera simulación. En esta caso llevandonos hacia el segundo camino que nos lleva al estado inicial `0000`.

Simulacion 3

Para esta tercera y ultima simulación exploramos el ultimo y más corto ciclo de la carta ASM, partiendo de las entradas iniciales `W=0,X=0, Y=0, Z=0` y modificando las entradas a `W=1,X=1, Y=1, Z=1`. La secuencia de estados y salidas es la siguiente:

sec-1: Estado: `0000` → Salida: `1111` → Estado siguiente: `0000` →

sec-2: Estado: `0000` → Salida: `0011` → Estado siguiente: `1001` →

sec-3: Estado: `1001` → Salida: `1000` → Estado siguiente: `0010` →

sec-4: Estado: `0010` → Salida: `0010` → Estado siguiente: `1000` →

sec-5: Estado: `1000` → Salida: `0111` → Estado siguiente: `0000` →

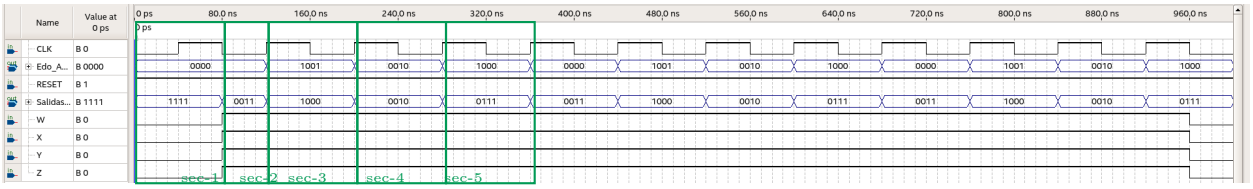


Figure 6: Simulación 1 de la máquina de estados en ModelSim, segmentada por transiciones de estado.

Notar que las entradas `Y=1` y `Z=1` no afectan el flujo de la máquina, ya que este ciclo depende del valor de la entrada `X` y de la evaluación de la entrada `W`.

Conclusiones

Andrea Saldaña Navarrete - 320206384

Hansel Yael Tepal Briseño - 320078282

Luis Antonio Ugartechea González - 320223822

1 Bibliografía

[1] D. K. Pérez Pérez, "Reporte Construcción de Máquinas de Estados Usando Memorias", Laboratorio de Organización y Arquitectura de Computadoras, Facultad de Ingeniería, Universidad Nacional Autónoma de México, Semestre 2027-1, 2026.

[2] Facultad de Ingeniería, "Capítulo III: Construcción de máquinas de estados usando memorias", Apuntes de Arquitectura de Computadoras, Universidad Nacional Autónoma de México (UNAM). [En línea]. Disponible en: https://biorobotics.fi-p.unam.mx/wp-content/uploads/Courses/arquitectura_de_computadoras/2021-1/lecciones/Leccion_3_arquitectura_de_computadoras_2021_1.pdf

[3] J. Savage, *Diseño de Microprocesadores*, Facultad de Ingeniería, Universidad Nacional Autónoma de México (UNAM). [En línea]. Disponible en: http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf